Polly Cracker

Concretions of A Combinatorial Algebraic Public Key Cryptosystem

Riki Hernandez

December 5, 2025

Abstract

In this paper, I analyze the public key cryptosystem Polly Cracker, introduced by Fellows and Koblitz. In particular, I will analyze the proposed graph-based special cases of Polly Cracker such as Graph 3-Coloring and Perfect Codes. In these schemes, the public key is the set of generators of an ideal in a polynomial ring, where the basis is derived from combinatorial or algebraic NP-complete problems and the secret key is a point where the ideal vanishes. Since a number of these schemes that were proposed in the textbook did not include concrete explanations, I will provide concretions and completed solutions to selected exercises and proofs that are only sketched in the textbook.[1] I will also touch upon subsequent cryptanalytic works and how they have shown that Polly Cracker and their variants are vulnerable to linear algebra attacks via worked examples, and why Polly Cracker has mostly failed in practice.

Contents

1 Introduction and History **2**
- 1.1 Historical Background and Context . . . 2
- 1.2 Notation and Terminology . . . 2
- 1.3 The Generic Polly Cracker Cryptosystem . . . 3

2 Special Cases of Polly Cracker **4**
- 2.1 Example 3.1: Graph 3-Coloring . . . 4
- 2.2 Example 3.2a: Graph Perfect Code . . . 5
- 2.3 Example 3.2: Subset Perfect Code . . . 6

3 Structural Vulnerabilities **8**
- 3.0.1 Exercise §3.7b: Attack over $\mathbb{F}_2$ . . . 8
- 3.0.2 Exercise §3.8: Attacks over $\mathbb{Z}$ and $\mathbb{Q}$ . . . 10
- 3.1 Gröbner Basis Attacks . . . 10
  - 3.1.1 Example: 4-Cycle with Cube Roots of Unity . . . 10

4 Conclusion **11**

1 Introduction and History

1.1 Historical Background and Context

In the 1990s, cryptographers were searching for alternatives to number-theoretic cryptosystems like RSA (based on integer factorization) or Diffie–Hellman (based on discrete logarithms). A natural idea was to use NP-hard combinatorial problems such as Graph 3-Coloring or the Knapsack problem as the basis for new public-key systems.

Polly Cracker is a representative of this approach in the algebraic setting. It relies on the *Ideal Membership Problem* for systems of multivariate polynomials over finite fields. For unrestricted instances this problem is known to be EXPSPACE-complete. The hope was that by encoding NP-complete problems as polynomial systems one could obtain trapdoor one-way functions.

Subsequent work by Barkee et al.[5], Hofheinz and Steinwandt[6], and others[4] has shown that the structured instances arising from graph-based constructions often do not behave like generic hard instances. In particular, they tend to leak enough information to make linear algebra attacks effective.

1.2 Notation and Terminology

We fix throughout a finite field $\mathbb{F}$ and consider the multivariate polynomial ring

$$R = \mathbb{F}[x_1, \ldots, x_n].$$

A (two-sided) ideal $I \subseteq R$ is a nonempty subset closed under addition and under multiplication by arbitrary elements of R. Given a finite set of polynomials $B = \{q_1, \ldots, q_m\} \subset R$, we write

$$J = \langle B \rangle$$

for the ideal generated by B. The *variety* of an ideal $J \subseteq R$, denoted $V(J)$, is the set of all points $\mathbf{y} \in \mathbb{F}^n$ at which every polynomial in J vanishes simultaneously. For any polynomial $f \in R$ and a fixed monomial ordering, we write $\mathrm{LT}(f)$ for the *leading term* of f with respect to that ordering.

The *Ideal Membership Problem* asks, given a finitely generated ideal $J = \langle B \rangle \subseteq R$ and a polynomial $f \in R$, whether $f \in J$. In full generality this problem is EXPSPACE-complete. We recall that EXPSPACE is the class of decision problems solvable by a deterministic Turing machine using at most $2^{p(n)}$ cells of work tape on inputs of length n, for some polynomial p.

A *trapdoor one-way function* is a function $F\colon X \to Y$ that is efficiently computable but computationally infeasible to invert on random inputs without auxiliary information, yet becomes efficiently invertible given some secret trapdoor data. True Cryptographic security requires that inversion be intractable in the average case for randomly sampled inputs, not merely in the worst case. A polynomial $f \in R$ is called *sparse* if it has relatively few nonzero terms (monomials) compared to the total number of possible terms of its degree, and *dense* if a large fraction of possible terms appear with nonzero coefficients. In the context of Polly Cracker, the choice between sparse and dense polynomials represents a fundamental tradeoff: sparse polynomials yield efficient encryption and compact ciphertexts but are vulnerable to linear algebra attacks that exploit the limited monomial support, while dense polynomials resist such structural attacks at the cost of prohibitively large key sizes and computational overhead. Throughout, when we refer to a finite simple graph $G = (V, E)$, we write $N[v]$ for the closed neighborhood of a vertex $v \in V$, consisting of v together with all vertices joined to v by an edge. For such a graph with $n = |V|$ vertices, we often index the vertices as $v_1, \ldots, v_n$ and collect labels c_v (or c'_v) into column vectors $\mathbf{c}, \mathbf{c}' \in \mathbb{F}^n$ (or $\mathbf{c}, \mathbf{c}' \in \mathbb{Z}^n$ in the integer variant). Given a graph G, we also use the matrix $A = (A_{v,u})_{v,u \in V}$ with entries

$$A_{v,u} = \begin{cases} 1, & \text{if } u \in N[v], \\ 0, & \text{otherwise,} \end{cases}$$

so that relations of the form $\mathbf{c}' = A\mathbf{c}$ capture the neighborhood-sum encryption rules in the

linear algebra attacks discussed in Section 3. For any vector $v \in \mathbb{F}^n$, we write $v^\top$ (or equivalently v^T) for its transpose, converting a column vector to a row vector. Finally, we write $\mathbf{1}$ for the all-ones column vector of length n, which is used to express message bits as sums of labels.

Remark. While we primarily work over finite fields $\mathbb{F}$ throughout this paper, certain exercises (notably §3.8) extend these constructions to the integer ring $\mathbb{Z}$ or the rational field $\mathbb{Q}$ to illustrate that the structural vulnerabilities persist across different algebraic settings.

1.3 The Generic Polly Cracker Cryptosystem

Fellows and Koblitz define Polly Cracker as a public key cryptosystem that can be phrased in terms of the Ideal Membership Problem for multivariate polynomials. The setup is defined over a finite field $\mathbb{F}$ and the polynomial ring

$$R = \mathbb{F}[x_1, \ldots, x_n].$$

Let $I \subseteq R$ be some fixed ideal. Messages are represented as elements of the quotient ring R/I. To encrypt a message class $m \in R/I$, one chooses a random element $f \in I$ and sends a representative

$$c = f + m \in R.$$

Decryption consists of computing the remainder of c modulo I, i.e., recovering its class in R/I, which returns m. Note that this is a probabilistic encryption since for the same message m, different random choices of f produce different ciphertexts c, all of which decrypt to the same message.

The Fellows and Koblitz's textbook describes the public key fomulation of Polly Cracker as follows:

1. **Key Generation:** Alice chooses a secret point (vector) $\mathbf{y} \in \mathbb{F}^n$. Her public key is a finite set of polynomials

   $$B = \{q_1, \ldots, q_m\} \subset R$$

   that all vanish at $\mathbf{y}$, i.e.

   $$q_j(\mathbf{y}) = 0 \quad \text{for all } j = 1, \ldots, m.$$

   Equivalently, the public ideal is $J = \langle B \rangle$, while the secret key is a point in the variety defined by J.

2. **Encryption:** To encrypt a message $m \in \mathbb{F}$, and supppose Bob chooses random "hiding" polynomials $h_j \in R$ and forms a polynomial

   $$p(\mathbf{x}) = \sum_{j=1}^{m} h_j(\mathbf{x}) q_j(\mathbf{x}) \in J.$$

   (Note that Bob's choice of the h_j is crucial, as it determines the vulnerabilities that will be described in Section 3.) Bob will then hide the scalar message m by sending the ciphertext polynomial

   $$c(\mathbf{x}) = p(\mathbf{x}) + m = \sum_{j=1}^{m} h_j(\mathbf{x}) q_j(\mathbf{x}) + m.$$

3. **Decryption:** Alice evaluates the ciphertext polynomial c at her secret point $\mathbf{y}$. Since every $q_j(\mathbf{y}) = 0$, the sum $p(\mathbf{y})$ vanishes and she recovers

   $$c(\mathbf{y}) = \sum_{j=1}^{m} h_j(\mathbf{y}) q_j(\mathbf{y}) + m = 0 + m = m.$$

Example 1.1. For simplicity, suppose Alice and Bob are working over the field $\mathbb{F}_2$, and Alice chooses the secret point $\mathbf{y} = (1, 0)$ and public polynomials

$$q_1 = x_1 + 1, \qquad q_2 = x_2.$$

These public polynomials both vanish at $\mathbf{y}$ (since $q_1(1,0) = 1 + 1 = 0$ and $q_2(1,0) = 0$).

To encrypt a message $m = 1$, suppose that Bob picks random hiding polynomials $h_1 = x_1x_2 + x_2$ (considered sparse with only 2 of 4 possible monomials $\{1, x_1, x_2, x_1x_2\}$) and $h_2 = x_1^2 + x_1 + 1$ (relatively dense with 3 out of 4 possible monomials). He then scrambles his message into a ciphertext by computing

$$\begin{aligned} p(\mathbf{x}) &= h_1 q_1 + h_2 q_2 \\ &= (x_1x_2 + x_2)(x_1 + 1) + (x_1^2 + x_1 + 1)x_2 \\ &= x_1^2x_2 + x_1x_2 + x_1x_2 + x_2 + x_1^2x_2 + x_1x_2 + x_2 \\ &= 0 \quad (\text{in } \mathbb{F}_2). \end{aligned}$$

The ciphertext is then

$$c = p(\mathbf{x}) + m = 0 + 1 = 1.$$

Alice can now receive $c = 1$ over a public channel and evaluates at $(1, 0)$:

$$c(1, 0) = 1,$$

Thus she successfully recovers the message. Note that Bob's choice of hiding polynomials with different sparsity patterns affects the structure of intermediate computations, though in this simple example the final ciphertext happens to be constant.

2 Special Cases of Polly Cracker

Fellows and Koblitz presents several graph theoretic constructions as special cases of Polly Cracker[1]. In each case, a classical NP-hard problem is encoded as an ideal membership problem by associating variables to combinatorial objects (vertices or subsets) and imposing polynomial relations that enforce the combinatorial constraints. Knowing a solution to the original problem corresponds to knowing a point where the ideal vanishes; this point serves as the private key, while the generating polynomials serve as the public key.

Note on formulations. The generic Polly Cracker system (Section 1.3) uses polynomial hiding: Bob chooses hiding polynomials h_j and constructs a ciphertext polynomial $c(\mathbf{x}) = \sum h_j(\mathbf{x})q_j(\mathbf{x}) + m$. However, for clarity and to facilitate visual representation, the encryption walkthroughs in the following examples employ a simplified scalar variant where variables are assigned numerical values described in exercise 7c in Koblitz's textbook[1]. Importantly, the structural vulnerabilities and cryptanalytic attacks discussed in Section 4 apply to both the standard polynomial formulation and these simplified scalar variants, as both rely on the same underlying linear algebraic structure.

In this section we recall three such constructions: Graph 3-Coloring, Subset Perfect Code, and Graph Perfect Code.

2.1 Example 3.1: Graph 3-Coloring

The objective is to construct a system of polynomials whose common zeros correspond exactly to a proper 3-coloring (private key) of a graph with unassigned vertices (public key) $G = (V, E)$. We work over a field $\mathbb{F}$ containing 0 and 1 (for simplicity one can take $\mathbb{F}_2$). For each vertex $v \in V$ and each color $i \in \{1, 2, 3\}$ we introduce a variable $t_{v,i}$, intended to be 1 if vertex v has color i and 0 otherwise. The public key consists of a basis $B = B(G)$ of polynomials in the variables $\{t_{v,i} : v \in V, 1 \leq i \leq 3\}$. This basis is given as $B = B_1 \cup B_2 \cup B_3$ where:

$B_1 = \{t_{v,1} + t_{v,2} + t_{v,3} - 1 : v \in V\}$. This forces, at any 0/1-valued solution, that exactly one of the three variables $t_{v,1}, t_{v,2}, t_{v,3}$ is equal to 1. Thus each vertex receives exactly one color.

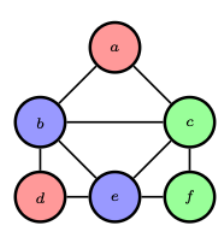
$B_2 = \{t_{v,i}t_{v,j} : v \in V, 1 \leq i < j \leq 3\}$. In the quotient ring $R/\langle B \rangle$, we have $t_{v,i}t_{v,j} = 0$ for all $i \neq j$. At a 0/1-valued point this means that it is impossible for two distinct color-indicator variables at the same vertex to be simultaneously 1, so no vertex can have two colors at once.

$B_3 = \{t_{u,i}t_{v,i} : uv \in E, 1 \leq i \leq 3\}$. These polynomials enforce the edge constraint: if $uv \in E$, then the relation $t_{u,i}t_{v,i} = 0$ forbids both $t_{u,i}$ and $t_{v,i}$ from being 1. Thus adjacent vertices cannot share the same color.

The zero set of the ideal $J = \langle B \rangle$ in $\{0, 1\}^{3|V|}$ is in bijection with the set of proper 3-colorings of G. This polynomial basis construction is given in Example 3.1 of [1].

Step 1: Key generation

Alice chooses a 3-coloring of public graph G. She publishes G and keeps the coloring secret.



Public: $G = (V, E)$. Private: coloring.

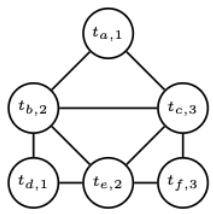
Step 2: Encryption

(Simplified variant, as note above.)

Bob sends bit $m \in \{0,1\}$ by assigning scalar values to variables $t_{v,i}$ satisfying the basis B such that $\sum_{v,i} t_{v,i} \equiv m \pmod 2$.

For $m = 1$: Bob sets variables corresponding to a valid coloring. For instance, he can set $t_{a,1} = 1$, $t_{b,2} = 1$, $t_{c,3} = 1$, $t_{d,1} = 1$, $t_{e,2} = 1$, $t_{f,3} = 1$, and all other $t_{v,i} = 0$. Then $\sum_{v,i} t_{v,i} = 6 \equiv 0 \pmod 2$.

To send $m = 1$, he adds one extra variable: set $t_{a,2} = 1$ as well (violating the coloring but still in the ideal's coset). Now $\sum_{v,i} t_{v,i} = 7 \equiv 1 \pmod 2$.



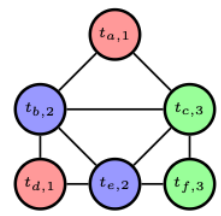
Bob's assignment: each vertex shows which $t_{v,i} = 1$ (color i).

Step 3: Decryption

Alice uses her secret coloring. For each vertex v with Alice's assigned color i, she reads $t_{v,i}$ and sums:

$$m = t_{a,1} + t_{b,2} + t_{c,3} + t_{d,1} + t_{e,2} + t_{f,3}.$$

With Bob's encoding, this gives the message bit m modulo 2.



Alice sums over her coloring.

2.2 Example 3.2a: Graph Perfect Code

Let $G = (V, E)$ be a finite simple graph (the public key). The private key is a perfect code, i.e. a subset $V' \subseteq V$ such that every vertex of V lies in the closed neighborhood $N[v]$ of exactly one $v \in V'$. Here $N[v]$ consists of v itself and all vertices joined to v by an edge.

We work with one variable t_v for each vertex $v \in V$, and consider polynomials in the ring $F[t_v : v \in V]$.

A basis $B = B_1 \cup B_2$ of polynomials in $\{t_v : v \in V\}$ is given by

$$B_1 = \left\{ 1 - \sum_{u \in N[v]} t_u : v \in V \right\},$$

and

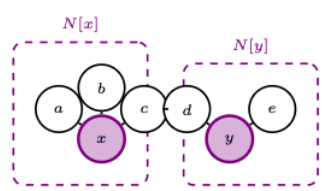
$$B_2 = \left\{ t_u t_{u'} : u, u' \in N[v],\ u \neq u',\ v \in V \right\}.$$

The relations in B_1 enforce that in each closed neighborhood $N[v]$ exactly one t_u equals 1, and B_2 forces that no two vertices in the same neighborhood are both chosen. This polynomial basis is described in Example 3.2a of [1].

Note: In the standard polynomial ideal formulation (Section 1.3), encryption instead constructs a ciphertext polynomial in the ideal $J = \langle B \rangle$. Despite this difference in representation, both approaches expose the same linear transformation $c' = Ac$ (as shown in Section 3), making them equally vulnerable to the linear algebra attacks.

Step 1: Key generation

Alice chooses a perfect code $V' \subseteq V$ of her public graph G. She publishes G as public key and keeps V' secret.

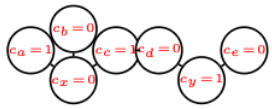


Public: G. Private: $V' = \{x, y\}$ (perfect code).

Step 2: Choosing raw labels

Bob sends message bit $b \in \{0,1\}$. He assigns bits c_v to vertices so that $\sum_{v \in V} c_v \equiv b \pmod 2$.

For $b = 1$: $(c_x, c_a, c_b, c_c, c_y, c_d, c_e) = (0,1,0,1,1,0,0)$, so $\sum c_v = 3 \equiv 1 \pmod 2$.



(b) All neighborhood parities c'_v.

Bob obtains the ciphertext labels c'_v by summing over each closed neighborhood $N[v]$.

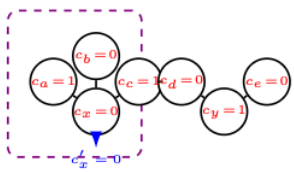
Step 3: Computing parities

For each vertex $v \in V$, Bob computes the neighborhood parity

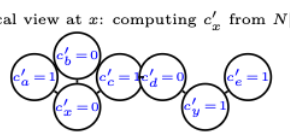
$$c'_v = \sum_{u \in N[v]} c_u \pmod 2.$$

For our choice of c_v, the neighborhoods and parities are:

$$\begin{aligned}
N[x] &= \{x, a, b, c\}, & c'_x &= 0 + 1 + 0 + 1 \equiv 0,\\
N[a] &= \{a, x\}, & c'_a &= 1 + 0 \equiv 1,\\
N[b] &= \{b, x\}, & c'_b &= 0 + 0 \equiv 0,\\
N[c] &= \{c, x, d\}, & c'_c &= 1 + 0 + 0 \equiv 1,\\
N[y] &= \{y, d, e\}, & c'_y &= 1 + 0 + 0 \equiv 1,\\
N[d] &= \{d, y, c\}, & c'_d &= 0 + 1 + 1 \equiv 0,\\
N[e] &= \{e, y\}, & c'_e &= 0 + 1 \equiv 1.
\end{aligned}$$

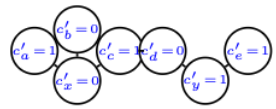


(a) Local view at x: computing c'_x from $N[x]$.



Step 4: Sending ciphertext

Bob sends Alice the graph with ciphertext bits c'_v. Raw labels c_v are never sent.

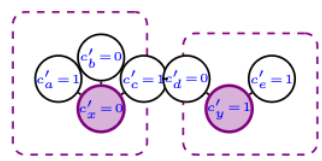


Step 5: Decryption

Using secret $V' = \{x, y\}$, Alice recovers b by summing over V':

$$b = \sum_{v \in V} c_v = \sum_{w \in V'} c'_w = c'_x + c'_y = 0 + 1 = 1 \pmod 2.$$

This works because $N[w]$ for $w \in V'$ partition V.



Alice uses $V' = \{x, y\}$: $b = c'_x + c'_y = 1 \pmod 2$.

2.3 Example 3.2: Subset Perfect Code

Let T be a finite set and $T_j \subset T$ (for $j = 1, \ldots, k$) be subsets such that $T = \bigcup_j T_j$. The public key consists of the set T and the subsets $\{T_j\}$. The private key is a subset $T_0 \subset T$ such that $T_0 \cap T_j$ consists of exactly one element for each $j = 1, \ldots, k$.

For each element $t \in T$, we introduce a polynomial variable (also denoted t) intended to be 1 if $t \in T_0$ and 0 otherwise.

A basis $B = B_1 \cup B_2$ of polynomials in the variables $t \in T$ is given by:

$$B_1 = \left\{ 1 - \sum_{t \in T_j} t : 1 \le j \le k \right\},$$

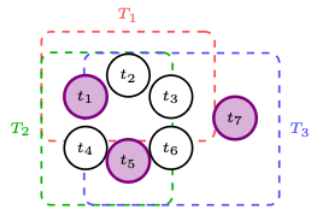
and

$$B_2 = \{\, tt' : t, t' \in T_j,\ t \neq t',\ 1 \le j \le k \,\}.$$

The relations in B_1 force exactly one variable in each T_j to be 1, ensuring each subset contains exactly one chosen element. The relations in B_2 prevent two distinct variables in the same T_j from both being 1, so no subset can have multiple chosen elements. This construction follows Example 3.2 in [1].

Step 1: Key generation

Alice chooses a perfect code $T_0 \subset T$ intersecting each subset T_j in exactly one element. She publishes T and $\{T_j\}$ as public key and keeps T_0 secret.

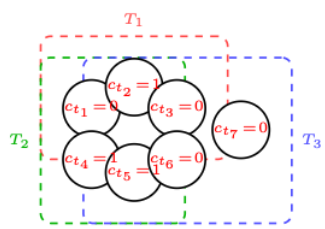


Public: $T, \{T_j\}$. Private: $T_0 = \{t_1, t_5, t_7\}$.

Step 2: Choosing raw labels

Bob sends message bit $b \in \{0,1\}$. He assigns bits c_t to elements so that $\sum_{t \in T} c_t \equiv b \pmod 2$.

For $b = 1$: $(c_{t_1}, c_{t_2}, c_{t_3}, c_{t_4}, c_{t_5}, c_{t_6}, c_{t_7}) = (0,1,0,1,1,0,0)$, so $\sum c_t = 3 \equiv 1 \pmod 2$.



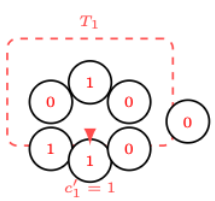
Step 3: Computing subset parities

For each subset T_j, Bob computes the subset parity

$$c'_j = \sum_{t \in T_j} c_t \pmod 2.$$

For our choice of c_t, the subset parities are:

$T_1 = \{t_1, t_2, t_3\}$, $\quad c'_1 = 0 + 1 + 0 \equiv 1$,
$T_2 = \{t_1, t_4, t_5\}$, $\quad c'_2 = 0 + 1 + 1 \equiv 0$,
$T_3 = \{t_3, t_5, t_6, t_7\}$, $\quad c'_3 = 0 + 1 + 0 + 0 \equiv 1$.



(a) Local view at T_1: computing c'_1.

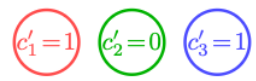
Ciphertext

(b) All subset parities c'_j.

Bob obtains ciphertext c'_j by summing over each subset T_j.

Step 4: Sending ciphertext

Bob sends Alice the subset parities c'_1, c'_2, c'_3. Raw labels c_t are never sent.



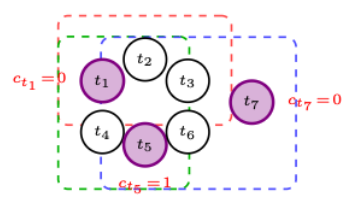
Ciphertext

Step 5: Decryption

Using secret $T_0 = \{t_1, t_5, t_7\}$, Alice knows which element belongs to each subset: $t_1 \in T_1 \cap T_0$, $t_5 \in T_2 \cap T_0$, and $t_7 \in T_3 \cap T_0$, so she reads Bob's raw values at these positions.

She recovers b by summing over elements in T_0:

$$b = \sum_{t \in T_0} c_t = c_{t_1} + c_{t_5} + c_{t_7} = 0 + 1 + 0 = 1 \pmod 2.$$



Alice uses T_0: $b = c_{t_1} + c_{t_5} + c_{t_7} = 1 \pmod 2$.

Exercises and Proofs

Exercise §3.1(a): In Examples 3.1, 3.2, and 3.2a, construct a one-to-one correspondence between private keys and points $\mathbf{y}$ at which the generating set B vanishes.

Proof. We sketch the argument for all three examples.

Graph 3-Coloring. Let $G = (V, E)$ and consider Example 3.1. A private key is a proper 3-coloring $C : V \to \{1,2,3\}$. For each vertex v, define $t_{v,i} = 1$ if $C(v) = i$ and $t_{v,i} = 0$ otherwise, and let $\mathbf{y}$ be the resulting point in $\{0,1\}^{3|V|}$. As explained above, the relations in B_1, B_2, B_3 are all satisfied by such an assignment, so $B(\mathbf{y}) = 0$. Conversely, if $\mathbf{y}$ is a 0/1-valued zero of B, then for each vertex exactly

one of $t_{v,1}, t_{v,2}, t_{v,3}$ equals 1 and no edge has both endpoints with the same color; interpreting the unique 1 in each triple as the color of that vertex yields a proper 3-coloring.

Subset Perfect Code. In Example 3.2, the public key is a family of subsets T_j of a set T, and the private key is a subset $T_0 \subseteq T$ meeting each T_j in exactly one element. We introduce variables t for $t \in T$, and the generators enforce that in each T_j exactly one t has value 1 and all pairwise products within T_j vanish. Assigning $t = 1$ if $t \in T_0$ and $t = 0$ otherwise produces a zero of the ideal; conversely, any 0/1-valued zero defines such a T_0.

Graph Perfect Code. In Example 3.2a, the public key is a graph $G = (V, E)$ and the private key is a perfect code $V' \subseteq V$. The variables t_v (for $v \in V$) and the generators in B_1 and B_2 ensure that in each closed neighborhood $N[v]$ there is exactly one vertex with $t_v = 1$ and that no two vertices in the same neighborhood are simultaneously chosen. Assigning $t_v = 1$ if $v \in V'$ and 0 otherwise yields a zero of the ideal, and conversely any 0/1-valued zero determines a perfect code. This completes the correspondence in all three cases. □

Exercise §3.1(b): In each of the three examples above, show that $t^2 - t$ belongs to the ideal $J = \langle B \rangle$ for each variable t.

Proof. Fix any example and let S be one of the blocks of variables appearing in a generator of type B_1:

- in Example 3.1, $S = \{t_{v,1}, t_{v,2}, t_{v,3}\}$ for some vertex v;
- in Example 3.2, $S = T_j$ for some j;
- in Example 3.2a, $S = N[v]$ for some $v \in V$.

In all cases, the ideal contains the linear polynomial

$$f_S = 1 - \sum_{u \in S} u \in B_1,$$

and all quadratic monomials uu' with distinct $u, u' \in S$ (these lie in B_2).

Fix any variable $t \in S$. Consider the combination

$$tf_S - \sum_{\substack{u \in S \\ u \neq t}} tu.$$

Expanding, we obtain

$$tf_S = t\left(1 - \sum_{u \in S} u\right) = t - t^2 - \sum_{\substack{u \in S \\ u \neq t}} tu,$$

so

$$tf_S - \sum_{u \neq t} tu = t - t^2 - 2\sum_{u \neq t} tu.$$

Each term on the left-hand side lies in J: tf_S is a multiple of $f_S \in B_1$, and each tu is one of the quadratic generators in B_2. Hence their difference is also in J, so

$$t - t^2 - 2\sum_{u \neq t} tu \in J.$$

But $2\sum_{u \neq t} tu \in J$ as a sum of generators, so subtracting it we obtain

$$t - t^2 \in J.$$

□

3 Structural Vulnerabilities

The security of Polly Cracker relies on the assumption that solving the Ideal Membership Problem or finding common zeros of a polynomial system is computationally intractable. However, the specific combinatorial structures used to generate keys such as graph neighborhoods or perfect codes introduce linear dependencies that allow attackers to bypass the intended algebraic hardness.[2] The Graph Perfect Code construction (Example 3.2a) demonstrates how sparsity allows ciphertext decryption via elementary linear algebra. Since the encryption is a linear transformation, an attacker can construct a decryption vector that bypasses the private key entirely.

3.0.1 Exercise §3.7b: Attack over $\mathbb{F}_2$

This attack exploits the linear structure of the simplified Graph Perfect Code variant described in Exercise §3.7 of [1].

We work with the Graph Perfect Code example from Example 3.2a. Let

$$V = \{x, a, b, c, y, d, e\}$$

with this fixed ordering of the vertices:

$$v_1 = x, \quad v_2 = a, \quad v_3 = b, \quad v_4 = c,$$
$$v_5 = y, \quad v_6 = d, \quad v_7 = e.$$

Assume that Alice's secret perfect code is

$$V' = \{x, y\} = \{v_1, v_5\}.$$

The closed neighborhoods are:

$$\begin{aligned} N[x] &= \{x, a, b, c\}, \\ N[a] &= \{a, x\}, \\ N[b] &= \{b, x\}, \\ N[c] &= \{c, x, d\}, \\ N[y] &= \{y, d, e\}, \\ N[d] &= \{d, y, c\}, \\ N[e] &= \{e, y\}. \end{aligned}$$

Let Bob's *raw* bits be the column vector

$$\mathbf{c} = (c_x, c_a, c_b, c_c, c_y, c_d, c_e)^\top \in \mathbb{F}_2^7.$$

He computes the *ciphertext* bits

$$\mathbf{c}' = (c'_x, c'_a, c'_b, c'_c, c'_y, c'_d, c'_e)^\top \in \mathbb{F}_2^7$$

by the neighborhood-summing rule

$$c'_v = \sum_{u \in N[v]} c_u \pmod 2.$$

This can be written as a matrix equation

$$\mathbf{c}' = A\,\mathbf{c} \quad \text{over } \mathbb{F}_2,$$

where the entries of the 7×7 matrix A are

$$A_{i,j} = \begin{cases} 1 & \text{if } v_j \in N[v_i], \\ 0 & \text{otherwise.} \end{cases}$$

With the chosen vertex ordering, A is

$$A = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

Bob's plaintext bit is

$$\begin{aligned} b &= \sum_{v \in V} c_v \\ &= c_x + c_a + c_b + c_c \\ &\quad + c_y + c_d + c_e \\ &= \mathbf{1}^\top \mathbf{c}, \end{aligned}$$

where $\mathbf{1} = (1, 1, 1, 1, 1, 1, 1)^\top$.

The legitimate decryption follows Alice using her secret perfect code $V' = \{x, y\}$ to recover b by computing $b = c'_x + c'_y$. Since the neighborhoods $N[x]$ and $N[y]$ partition V, summing the ciphertext components over V' reconstructs $\sum_{v \in V} c_v = b$.

To find a decryption vector, attacker sees only the public matrix A and the ciphertext $\mathbf{c}'$, but not the raw vector $\mathbf{c}$ or the secret code V'. The key observation is that *any* vector $w \in \mathbb{F}_2^7$ satisfying

$$w^\top A = \mathbf{1}^\top$$

allows recovery of the message via

$$\begin{aligned} w^\top \mathbf{c}' &= w^\top (A\mathbf{c}) \\ &= (w^\top A)\mathbf{c} \\ &= \mathbf{1}^\top \mathbf{c} = b. \end{aligned}$$

Hence, the private key is superfluous and only the public linear structure matters.

We can verify an explicity solution from our previous example by taking

$$w = (1, 0, 0, 0, 1, 0, 0)^\top,$$

i.e. we sum the x-row and the y-row of A.

$$\begin{aligned} w^\top A &= \mathrm{row}_x(A) + \mathrm{row}_y(A) \\ &= (1, 1, 1, 1, 0, 0, 0) \\ &\quad + (0, 0, 0, 0, 1, 1, 1) \\ &= (1, 1, 1, 1, 1, 1, 1) \\ &= \mathbf{1}^\top \text{in } \mathbb{F}_2^7. \end{aligned}$$

Therefore,

$$b = w^\top \mathbf{c}' = c'_x + c'_y,$$

matching Alice's decryption formula exactly. The scheme is completely broken by solving $w^\top A = \mathbf{1}^\top$ over $\mathbb{F}_2$.

3.0.2 Exercise §3.8: Attacks over $\mathbb{Z}$ and $\mathbb{Q}$

This integer variation of the Graph Perfect Code system is described in Exercise §3.8 of [1].

The attack remains identical when the protocol operates over the integers rather than $\mathbb{F}_2$. Bob chooses integers

$$\mathbf{c} = (c_x, c_a, c_b, c_c, c_y, c_d, c_e)^\top \in \mathbb{Z}^7$$

such that their sum

$$m := \sum_{v \in V} c_v = \mathbf{1}^\top \mathbf{c} \in \mathbb{Z}$$

is the message. He then computes

$$\mathbf{c}' = A\,\mathbf{c} \in \mathbb{Z}^7$$

with the same neighborhood matrix A as above, and sends $\mathbf{c}'$ to Alice.

Alice's decryption rule is $m = c'_x + c'_y$.

An attacker can solve $A^\top w = \mathbf{1}$ over $\mathbb{Q}$ (or over $\mathbb{Z}$). Any solution w_0 yields

$$\begin{aligned} m &= \mathbf{1}^\top \mathbf{c} \\ &= w_0^\top A\mathbf{c} \\ &= w_0^\top \mathbf{c}', \end{aligned}$$

directly recovering the message. In our example,

$$w_0 = (1, 0, 0, 0, 1, 0, 0)^\top$$

satisfies $A^\top w_0 = \mathbf{1}$ as an integer equation, giving $m = w_0^\top \mathbf{c}' = c'_x + c'_y$ exactly as before.

3.1 Gröbner Basis Attacks

Even when linear algebra fails, structured polynomial ideals often succumb to Gröbner basis algorithms. We demonstrate this on a Graph 3-Coloring instance (Example 3.1) scaled to a 4-cycle and show how finding any zero of the public ideal functions as a valid decryption key.

3.1.1 Example: 4-Cycle with Cube Roots of Unity

Consider a 4-cycle using the cube roots of unity (to keep the Gröbner basis small).
Let $G = (V, E)$ be the 4-cycle with $V = \{v_1, v_2, v_3, v_4\}$ and $E = \{v_1v_2, v_2v_3, v_3v_4, v_4v_1\}$, over a field F that contains three distinct cube roots of unity $\{1, \omega, \omega^2\}$ (so $\operatorname{char} F \neq 3$).

Introduce one variable x_v for each vertex $v \in V$:

$$x_1 := x_{v_1}, \quad x_2 := x_{v_2}, \quad x_3 := x_{v_3}, \quad x_4 := x_{v_4}.$$

Following the alternative encoding for 3-coloring using cube roots of unity (Exercise §3.3 in [1], which requires $\operatorname{char} F \neq 3$ to ensure distinct roots), we define

$$B' = B'_1 \cup B'_2 \subset F[x_1, x_2, x_3, x_4],$$

where

$$B'_1 = \{\, x_i^3 - 1 : i = 1, 2, 3, 4 \,\}$$

enforces that each x_i is a cube root of unity, and

$$B'_2 = \{\, x_u^2 + x_u x_v + x_v^2 : uv \in E \,\}$$

enforces that endpoints of an edge do not share the same color. Explicitly, for the 4-cycle:

$$\begin{aligned} B'_1 = \{\, & x_1^3 - 1,\ x_2^3 - 1, \\ & x_3^3 - 1,\ x_4^3 - 1 \,\}, \end{aligned}$$

$$\begin{aligned} B'_2 = \{\, & x_1^2 + x_1x_2 + x_2^2, \\ & x_2^2 + x_2x_3 + x_3^2, \\ & x_3^2 + x_3x_4 + x_4^2, \\ & x_4^2 + x_4x_1 + x_1^2 \,\}. \end{aligned}$$

Let J be the ideal

$$J := \langle B' \rangle \subset F[x_1, x_2, x_3, x_4].$$

Now consider the lexicographic term order

$$x_1 > x_2 > x_3 > x_4.$$

Computing a Gröbner basis of J with respect to this order yields a reduced Gröbner basis

$$G = \{g_1, g_2, g_3, g_4, g_5\}$$

given by:

$$\begin{aligned} g_1 &= x_1^2 + x_1x_4 + x_4^2, \\ g_2 &= x_1x_2 - x_1x_4 - x_2x_3 + x_3x_4, \\ g_3 &= x_2^2 + x_2x_3 - x_3x_4 - x_4^2, \\ g_4 &= x_3^2 + x_3x_4 + x_4^2, \\ g_5 &= x_4^3 - 1. \end{aligned}$$

The leading terms (with the chosen lex order) are

$$\begin{aligned}\mathrm{LT}(g_1) &= x_1^2,\\ \mathrm{LT}(g_2) &= x_1x_2,\\ \mathrm{LT}(g_3) &= x_2^2,\\ \mathrm{LT}(g_4) &= x_3^2,\\ \mathrm{LT}(g_5) &= x_4^3.\end{aligned}$$

These monomials are pairwise non-dividing, and one checks that G is indeed a reduced Gröbner basis of J.

Here, notice that the Gröbner basis G is essentially triangular:

1. g_5 constrains x_4 to be a cube root of unity: $x_4^3 = 1$.
2. Given x_4, the equations g_4 and g_1 are quadratic relations that determine x_3 and x_1 among the cube roots of unity.
3. The equation g_3 similarly constrains x_2 given x_3 and x_4.
4. Finally, g_2 ties together (x_1, x_2, x_3, x_4) and excludes colorings that do not come from proper 3-colorings of the 4-cycle.

In particular, the zero set of J consists exactly of the proper 3-colorings of G, encoded as tuples (x_1, x_2, x_3, x_4) of cube roots of unity. Any solution of $G = 0$ provides a valid private key (a proper 3-coloring), and conversely every proper 3-coloring gives a solution.

An attacker who can compute a Gröbner basis G of J can solve $G = 0$ by back-substitution to enumerate all proper 3-colorings of the 4-cycle. Furthermore, any of these solutions can be used as a substitute private key to decrypt ciphertexts. Since ciphertexts $c(\mathbf{x})$ evaluate to the message m at any point in the variety $V(J)$, finding any zero via Gröbner bases suffices. For structured combinatorial ideals, such computations are often feasible and the one-way function collapses completely. The secret trapdoor is irrelevant.

4 Conclusion

The original special cases of Polly Cracker proposed by Fellows and Koblitz, which relied on NP-complete combinatorial problems such as Graph 3-Coloring and Graph Perfect Code [1] along with the Generalized Gröbner Basis schemes [2] have largely failed because they relied on sparse polynomials or hidden monomials to disguise the secret key [5, 6]. Attacks such as the intelligent linear algebra attack and the differential attack exploit this sparsity [6]. Even attempts to hide characteristic monomials proved insecure, as differential analysis can reveal them and allow attackers to reconstruct the plaintext without knowing the private key [6]. Consequently, essentially all proposed variants of the original Polly Cracker framework have been defeated [3, 4]. However, despite these failures, recent proposals that involve shifting the hardness assumption to lattice-based problems have shown that it may still be possible to construct secure instances that resist the differential attack[4].

Works Cited

References

[1] Neal Koblitz. *Algebraic Aspects of Cryptography.* Springer-Verlag, 1998.

[2] Christoph Strobl. How Is Polly? Revisiting the Differential Attack on Polly Cracker After 20 Years. Master's Thesis, Faculty of Science, Centre for Mathematical Sciences, Mathematics, 2022.

[3] Martin Albrecht, Pooya Farshim, Jean-Charles Faugère, and Ludovic Perret. Polly Cracker, Revisited. Inria / UPMC / TU Darmstadt (full version).

[4] Massimo Caboara, Fabrizio Caruso, and Carlo Traverso. Lattice Polly Cracker Cryptosystems. *Journal of Symbolic Computation*, vol. 46, 2011, pp. 534–549.

[5] Barkee Barkee, D. C. W. Can, J. Ecks, T. Moriarty, and R. F. Ree. Why you cannot even hope to use Gröbner bases in public key cryptography: an open letter to a scientist who failed and the others who might. *Journal of Symbolic Computation*, vol. 18, no. 6, 1994, pp. 497–501.

[6] Dennis Hofheinz and Rainer Steinwandt. A "Differential" Attack on Polly Cracker. In *Proceedings of the International Conference on Information Security and Cryptology*, 2002.